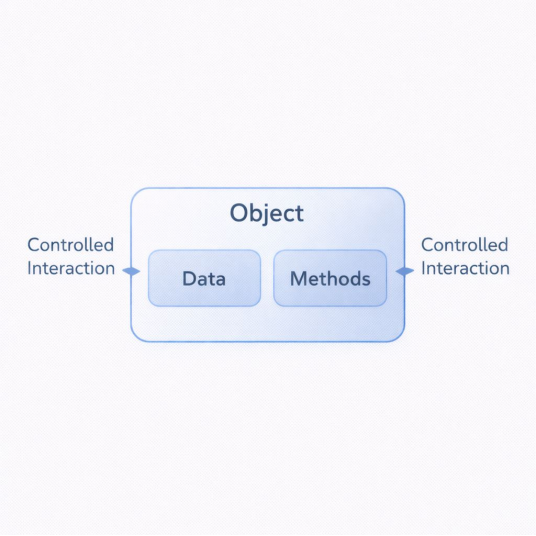
Python Programming Crash

Managed Attributes and Decorators

OOP Recap — Objects, Classes, and Responsibility

Objects are not just data containers — they are responsible for maintaining valid state and behavior.

- Objects combine **data (attributes) + behavior (methods)**
- Classes define structure and rules for objects
- Objects should protect their own state
- Direct access to attributes can break consistency
- Good design = **controlled access + clear responsibility**



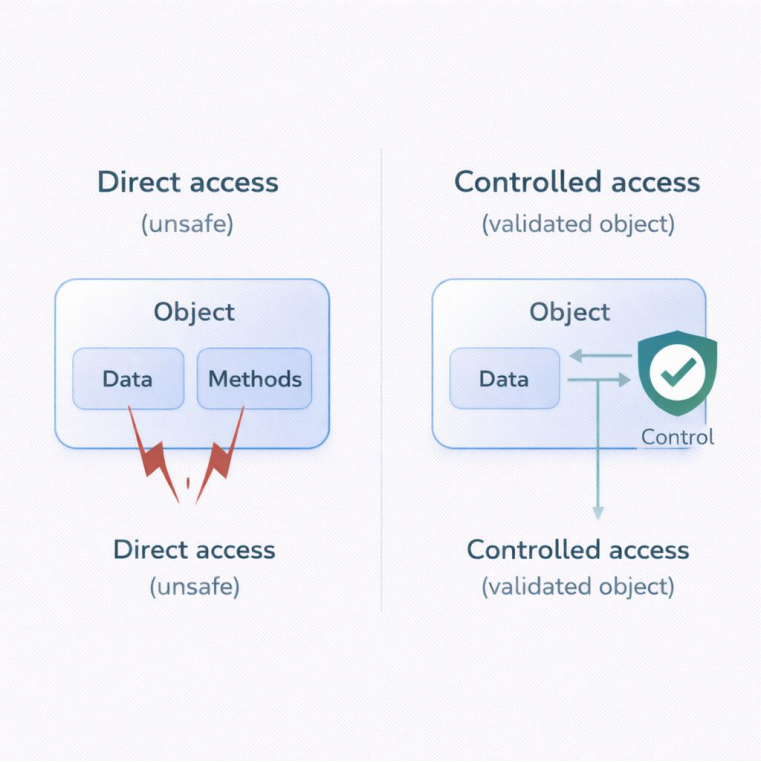
Why Attribute Control Matters

Uncontrolled attribute access leads to bugs, inconsis

- Direct attribute access allows invalid data
- Validation logic becomes scattered
- Hard to maintain in large systems
- Breaks encapsulation principles
- Leads to fragile and error-prone code

order.total = -500

→ “Where should validation happen?



Why Manage Attributes?

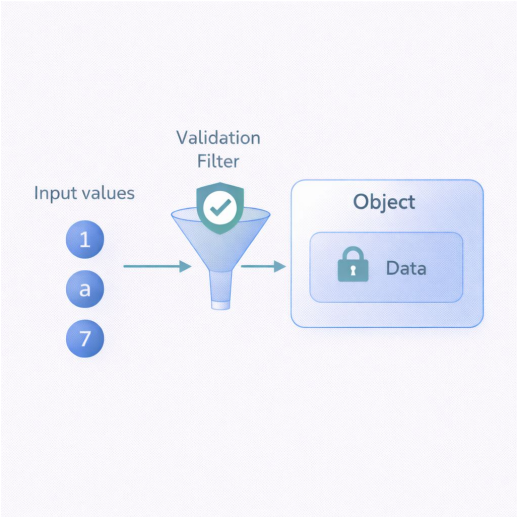
Control how data is accessed and modified to ensure reliability, security, and maintainability.

- Prevent invalid or inconsistent data through validation
- Hide internal implementation using encapsulation
- Control how data is modified (controlled mutation)
- Enforce business rules at the object level
- Foundation for scalable and maintainable systems

Data Validation

Ensure only valid data is assigned to attributes.

- Prevent invalid values (e.g., negative age, empty
- Enforce constraints at the object level
- Reduce bugs early in the system
- Centralize validation logic
- Improve data integrity across applications



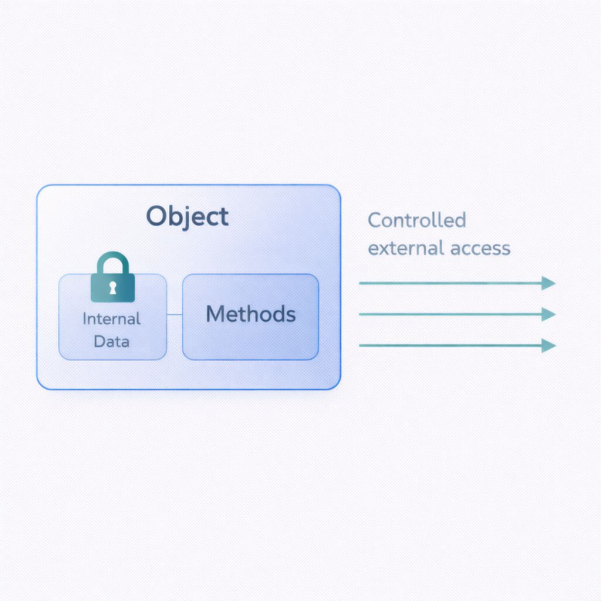
- Demo example:

```
if age < 0:
   raise ValueError("Age cannot be negative")
```

Encapsulation

Hide internal data and expose only what is necessary.

- Restrict direct access to internal attributes
- Use methods or properties for controlled access
- Protect internal state from unintended changes
- Improve code readability and maintainability
- Core principle of object-oriented design



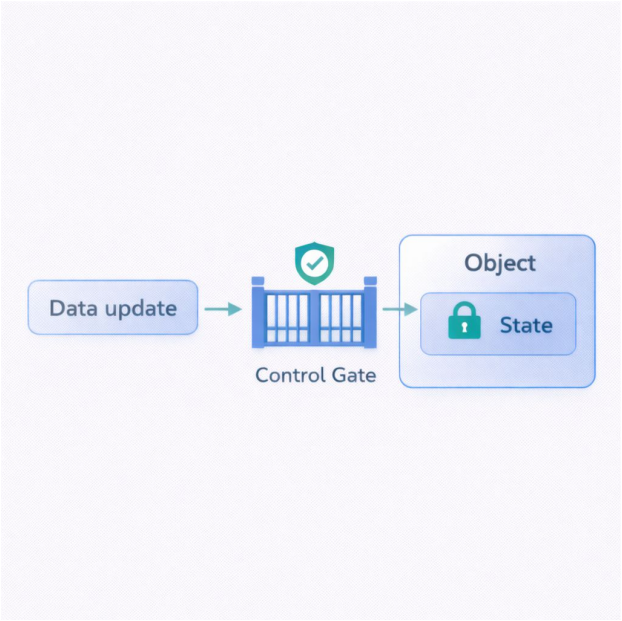
Example:

```
self._balance   # internal use
```

Controlled Mutation

Allow data changes only through defined and safe operations.

- Prevent arbitrary modification of attributes
- Enforce rules when updating values
- Use setters or properties for control
- Maintain consistency across state changes
- Supports auditability and debugging



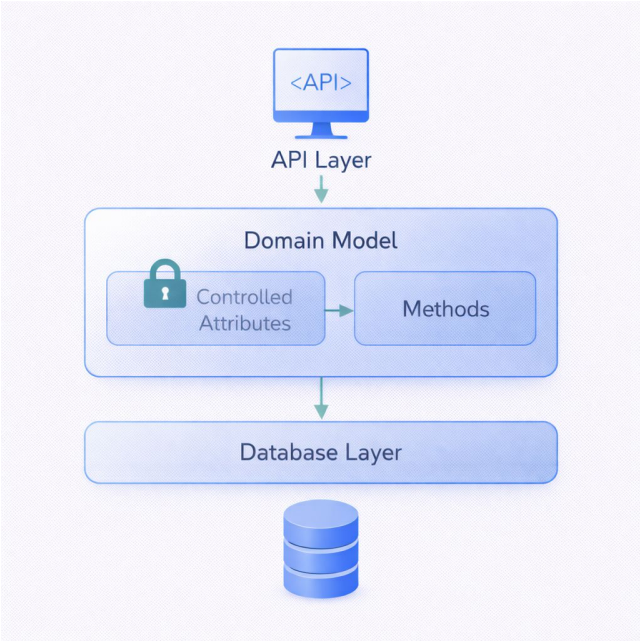
Example:

```
def set_age(self, age):
    if age < 0:
        raise ValueError("Invalid age")
    self._age = age
```

Enterprise Design Patterns

Managed attributes are the foundation of robust enterprise systems.

- Used in domain-driven design (DDD)
- Supports clean architecture principles
- Ensures consistency across large systems
- Enables reusable and testable components
- Common in APIs, services, and data models



Introduction to Properties

Use `@property` to control attribute access like a method while keeping simple syntax.

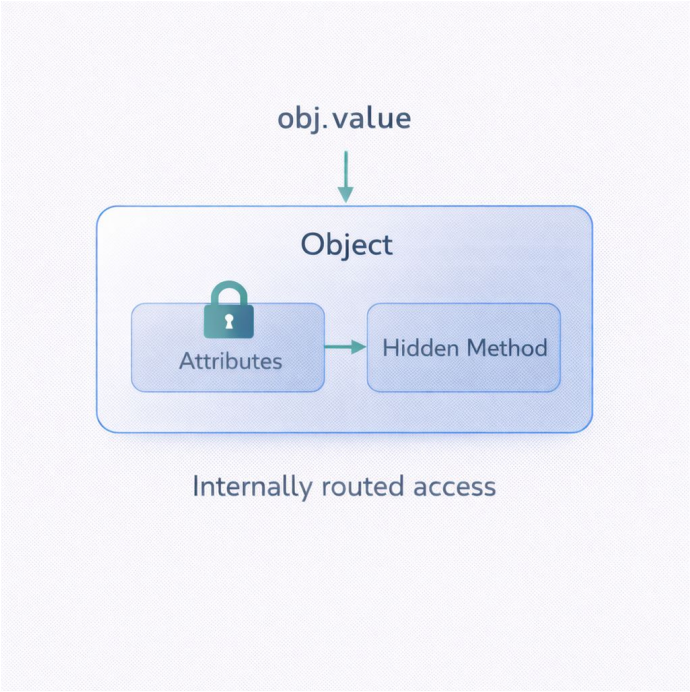
- `@property` allows method access like an attribute
- Enables **controlled read access** to internal data
- Keeps clean and intuitive API (`obj.value` instead of `obj.get_value()`)
- Supports encapsulation without changing usage syntax
- Foundation for getters, setters, and validation

Demo @property

```
class User:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name
```

Usage:

```
user.name   # looks like attribute, actually method
```



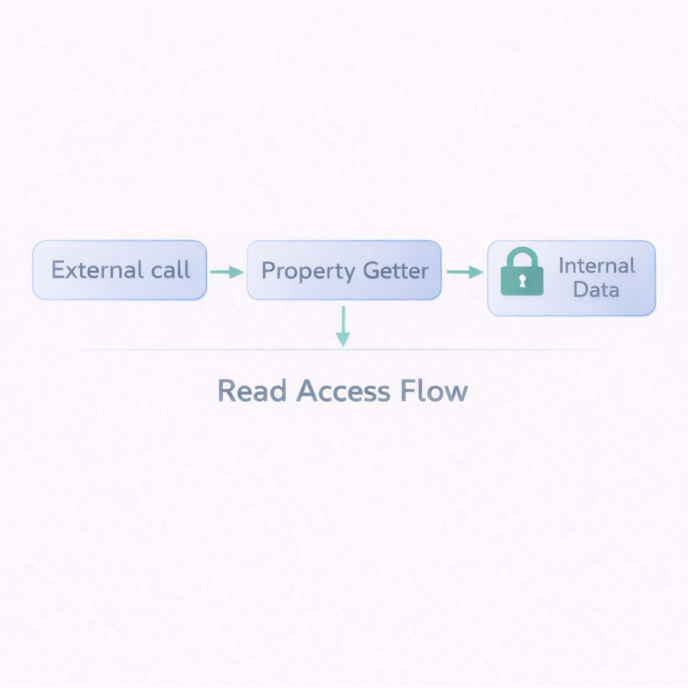
Getter with @property

Define how an attribute is accessed using a property getter.

- Getter method is defined using `@property`
- Returns internal attribute value
- Can include computed logic if needed
- Prevents direct access to internal variables
- Improves readability and abstraction

Example:

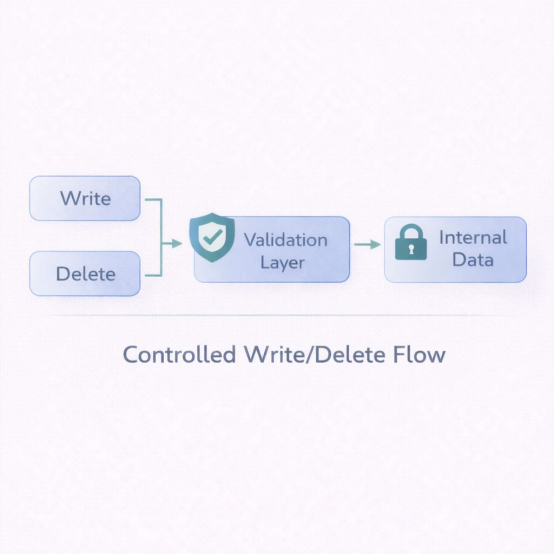
```
@property
def age(self):
    return self._age
```



Setter & Deleter

Control how attributes are modified or removed using property decorators.

- Use `@<property>.setter` to control assignment
- Use `@<property>.deleter` to control deletion
- Enables validation before updating values
- Prevents invalid or unsafe modifications
- Keeps logic centralized in one place



Example Setter & Deleter

```
 @age.setter
def age(self, value):
    if value < 0:
        raise ValueError("Age cannot be negative")
    self._age = value
```

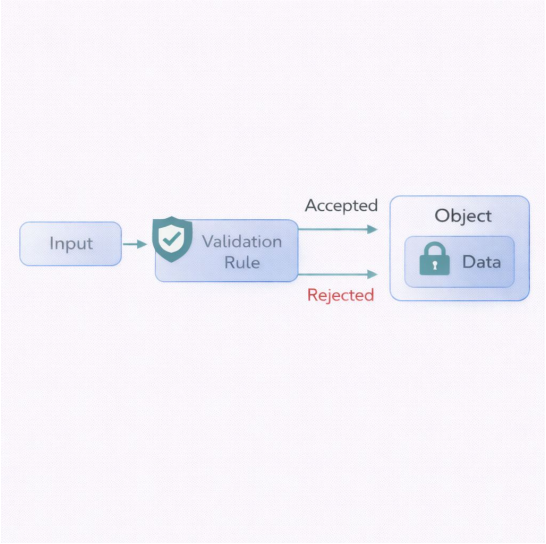
Deleter:

```
 @age.deleter
def age(self):
    del self._age
```

Validation Patterns with Properties

Use properties to enforce business rules and ensure data integrity.

- Validate inputs before assigning values
- Raise exceptions for invalid data
- Enforce rules at the object level
- Avoid duplicated validation logic
- Common pattern in real-world systems



Example Validation Patterns:

```
 @property
def email(self):
    return self._email

@email.setter
def email(self, value):
    if "@" not in value:
        raise ValueError("Invalid email")
    self._email = value
```

Why Properties Matter

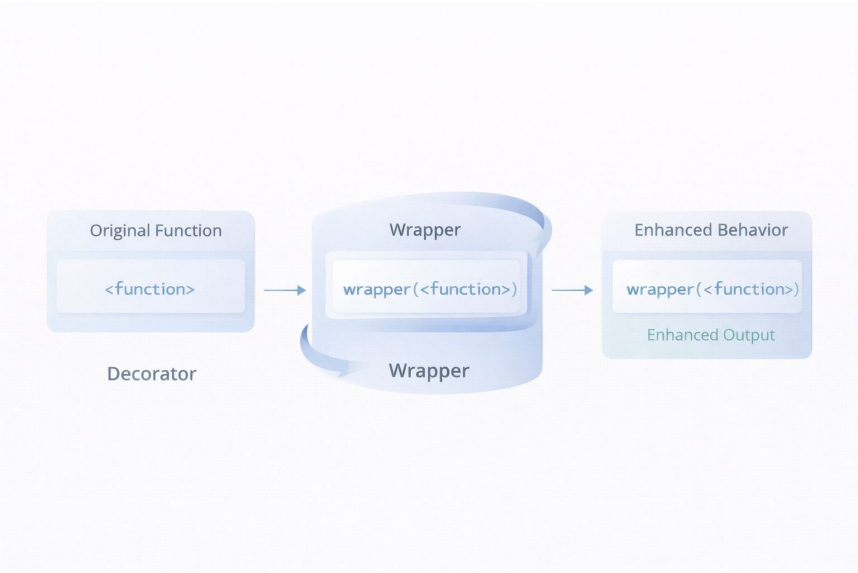
Properties provide clean syntax with powerful control, making them essential for modern Python design.

- Combine simplicity of attributes with power of methods
- Improve code readability and maintainability
- Enable safe and controlled data access
- Align with object-oriented best practices
- Widely used in frameworks and enterprise systems

Lab 5: Decorators Deep Dive — Part 1: Function Decorators

Build a solid foundation in Python function decorators by learning how to:

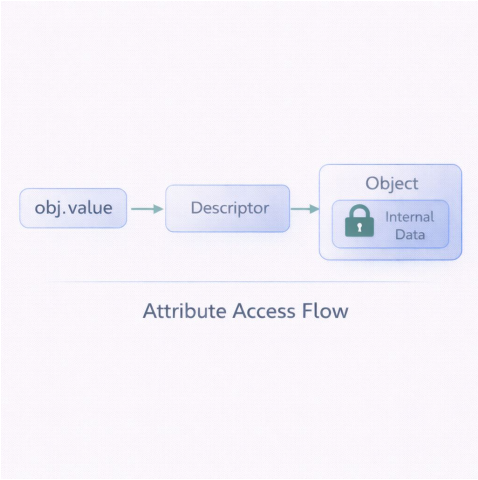
- create a basic decorator
- wrap and enhance existing functions
- preserve original function metadata
- use `functools.wraps` correctly



What Are Descriptors?

Descriptors are objects that control attribute access using reusable logic.

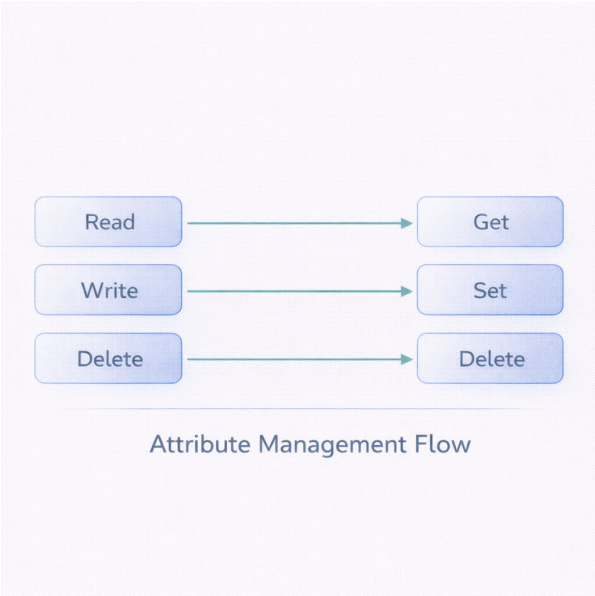
- A descriptor is a class that defines `__get__`, `__set__`, or `__delete__`
- Used to manage how attributes are accessed and modified
- Powers Python features like `@property` internally
- Enables reusable and centralized attribute logic
- Common in frameworks and advanced Python design



Descriptor Methods: **get**, **set**, **delete**

Descriptors control read, write, and delete operations through special methods.

- `__get__` handles attribute access
- `__set__` handles attribute assignment
- `__delete__` handles attribute deletion
- Each method intercepts operations on attributes
- Enables full lifecycle control of data



Example: Descriptor Methods

```
class MyDescriptor:
    def __get__(self, instance, owner):
        return instance._value

    def __set__(self, instance, value):
        instance._value = value
```

Basic Descriptor Example

Implement custom logic for attribute access using descriptors.

- Descriptor is assigned as a class attribute
- Automatically intercepts attribute operations
- Stores or retrieves values from instance
- Can enforce rules or transformations
- Acts like a reusable property

Demo Basic Descriptor Example

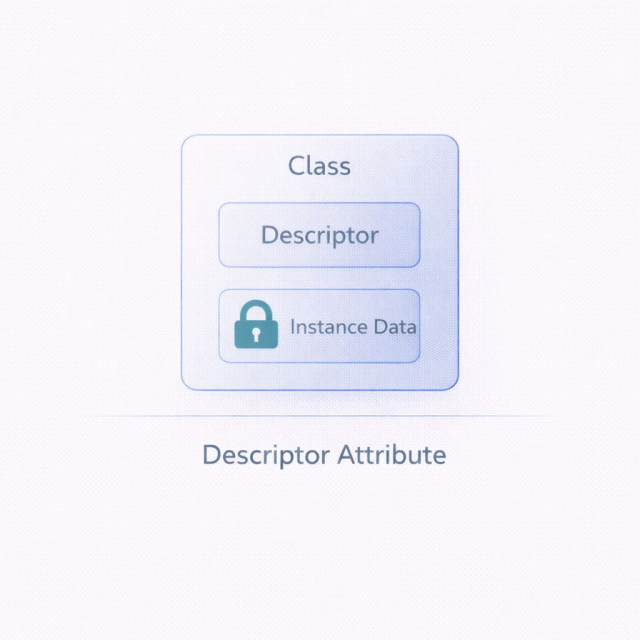
```
class PositiveNumber:
    def __get__(self, instance, owner):
        return instance._value

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("Must be positive")
        instance._value = value

class Product:
    price = PositiveNumber()
```

```
p = Product()
p.price = 100
```

Reusable Validation Descriptors

Create reusable validation logic that can be applied across multiple classes.

- Encapsulate validation in a separate class
- Reuse across multiple attributes and models
- Avoid duplicated validation code
- Improve consistency and maintainability
- Useful in enterprise and framework design



Example: Reusable Validation Descriptors

```
class NonEmptyString:
    def __set__(self, instance, value):
        if not value:
            raise ValueError("Cannot be empty")
        instance._name = value
```

Apply to multiple classes:

```
class User:
    name = NonEmptyString()

class Product:
    name = NonEmptyString()
```

Descriptors vs Properties

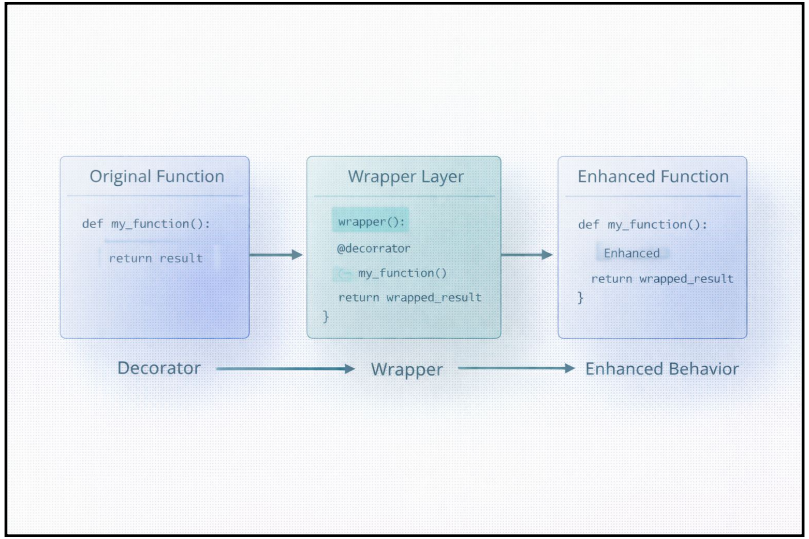
Descriptors offer more flexibility and reusability compared to properties.

- Properties are simpler and built-in
- Descriptors are more powerful and reusable
- Properties are tied to one class
- Descriptors can be shared across classes
- Use descriptors for advanced, scalable systems

Lab 6 — Function Decorator for Rate Limiting

Build a Python decorator that:

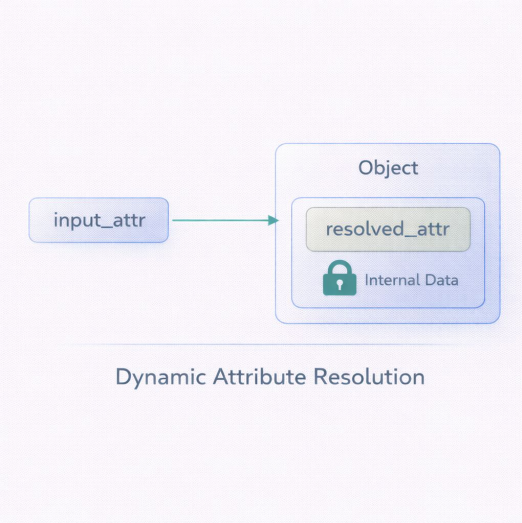
- limits how many times a function can be called per minute
- logs each request timestamp
- raises an exception when the limit is exceeded



Dynamic Attribute Access

Use `getattr` and `__getattribute__` to access and control attributes dynamically at runtime.

- `getattr(obj, name)` retrieves attributes dynamically
- Useful when attribute names are **not known at compile time**
- Supports default values to prevent errors
- Enables flexible and dynamic code design
- Common in frameworks, APIs, and dynamic data handling



Example: Dynamic Attribute Access

getattr(user, "name")

- With default:

  getattr(user, "age", 0)

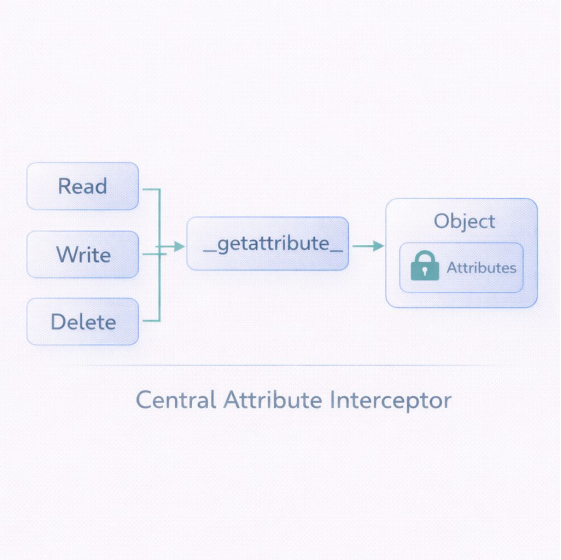
Use case:
→ dynamic fields from JSON or config

Emphasize:
→ safer than direct attribute access

getattribute Overview

`__getattribute__` intercepts every attribute access for full control.

- Called automatically on **every attribute access**
- Provides complete control over attribute retrieval
- Can modify or restrict access dynamically
- Must be used carefully to avoid infinite recursion
- Core mechanism behind Python attribute access



Example getattribute

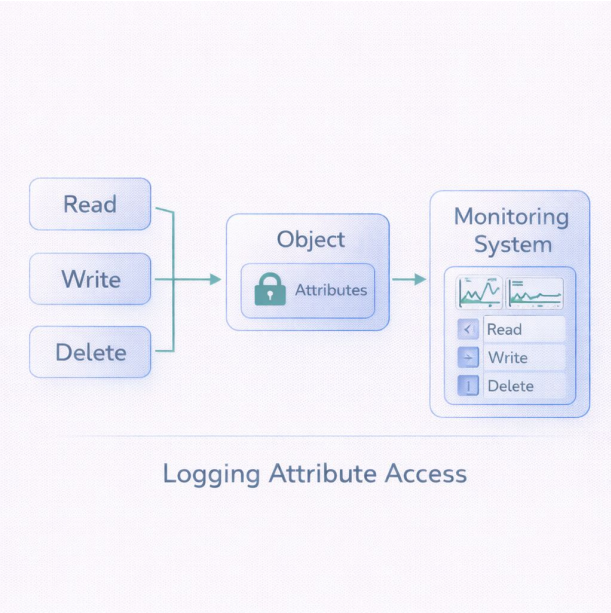
```
class User:
    def __getattribute__(self, name):
        print(f"Accessing {name}")
        return super().__getattribute__(name)
```

- Key concept:
  → every `obj.attr` goes through this method
- Warning:
  → always call `super()` to avoid recursion

Logging Attribute Access

Track attribute usage for debugging, monitoring, and auditing.

- Log every attribute read operation
- Useful for debugging complex systems
- Helps trace unexpected behavior
- Can be extended for audit logging
- Common in enterprise and security systems



Example Logging Attribute Access

```
class User:
    def __getattribute__(self, name):
        print(f"[LOG] Accessing: {name}")
        return super().__getattribute__(name)
```

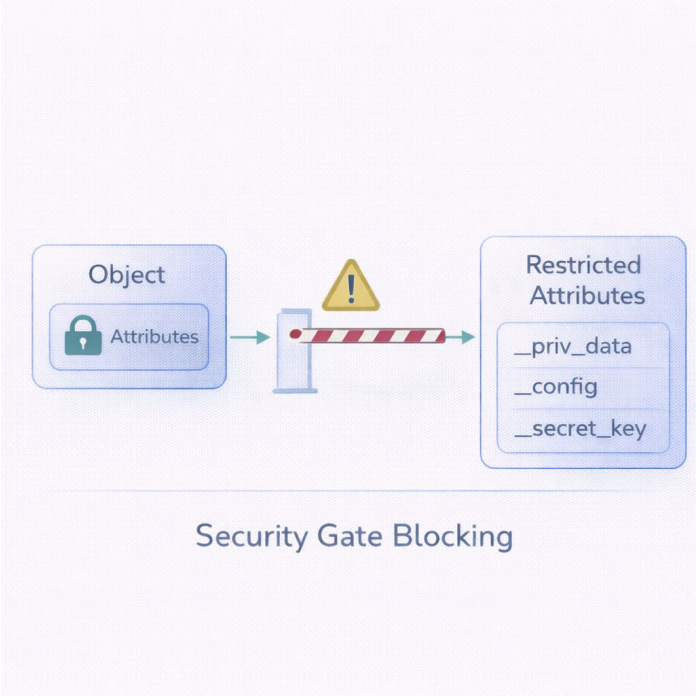
Explaination:
→ every access is now visible

Real-world:
→ audit logs, security tracking

Preventing Unsafe Access

Restrict access to sensitive attributes using dynamic control.

- Block access to private or sensitive attributes
- Enforce security rules at the object level
- Prevent unintended or malicious access
- Raise exceptions for restricted fields
- Strengthen application-level security



Example: Preventing Unsafe Access

```
class User:
    def __getattribute__(self, name):
        if name == "password":
            raise AttributeError("Access denied")
        return super().__getattribute__(name)
```

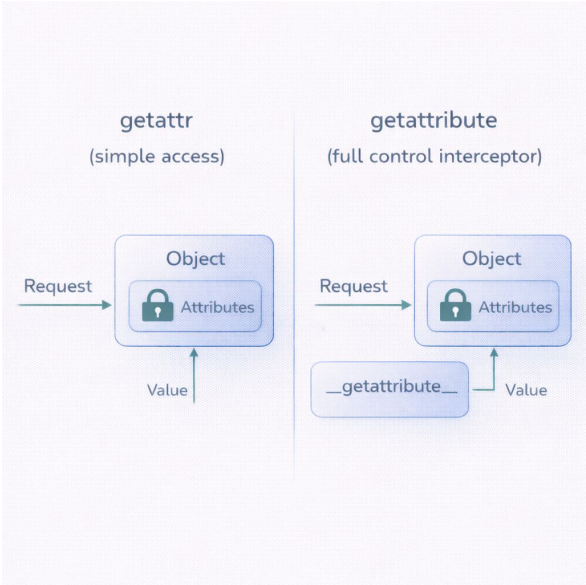
Emphasize:
→ object enforces its own security

Real-world:
→ hiding sensitive data (passwords, tokens)

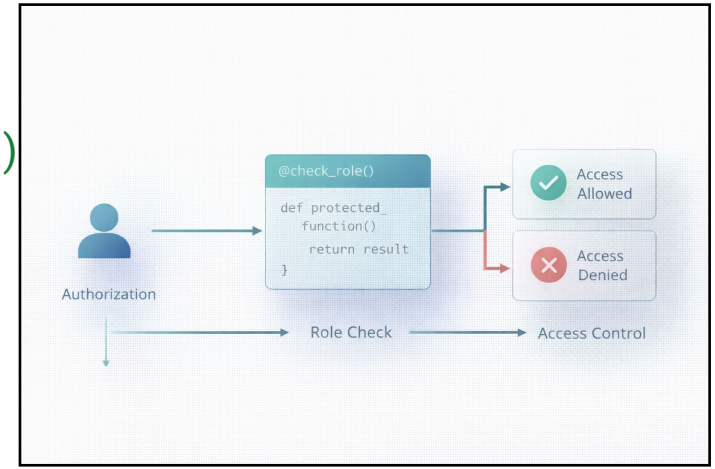
getattr vs **getattribute**

Understand when to use simple dynamic access vs full control interception.

- `getattr` is simple and safe for dynamic access
- `__getattribute__` provides full control but is complex
- `getattr` is used externally
- `__getattribute__` is used inside class design
- Choose based on complexity and control needs



Lab 7 — Authorization Decorator (`@requires_role`)

Build a reusable Python decorator that:

- restricts function execution based on user role
- supports syntax like `@requires_role("admin")`
- logs access attempts (allowed/denied)
- raises a clear exception when authorization fails